Numerical Analysis: A Tourist's Guide

Cooper Hince

25/09/26

Abstract

This is a short write-up for one particular method of solving differential equations numerically, and then applied to a couple real world examples. All mathematics was done by hand initially, then transcribed into this document and polished for ease of readability and understanding.

The main objective is in deriving a method for simulating the three body problem in Python, with some other unrelated applications and ideas throughout. Truncated error and energy conservation are not covered in this document. This is just a basic implementation of some numerical ideas, hence the title, “A Tourist’s Guide”.

The first chapter derives some theorems, and the second chapter applies those theorems to some real world applications.

Contents

1 | Numerical Analysis for Ordinary Differential Equations 4
1.1 | Forward finite difference recursion for x,y explicit *nth* order ODEs 4
1.1.1 | Derivation of x,y forward recursion 4
1.1.2 | Derivation of initial conditions 5
1.1.3 | First order ODE recursion 5
1.1.4 | Second order ODE recursion 5
1.1.5 | Third order ODE recursion 5
1.2 | Forward finite difference recursion for general explicit *nth* order ODEs 6
1.2.1 | Derivation of general forward recursion 6
1.2.2 | Second order general ODE recursion 6
2 | Real World Differential Equations 7
2.1 | The *n* body problem 7
2.1.1 | Derivation of the *n* body problem coupled system 7
2.1.2 | The discrete three body problem 8
2.1.3 | Simulating the three body problem 9
2.2 | Pendulum dynamics 10
2.2.1 | The discrete damped pendulum 10

1 | Numerical Analysis for Ordinary Differential Equations

This chapter will explore how a basic form of numerical analysis can be used to approximate solutions to ordinary differential equations (ODEs).

1.1 | Forward finite difference recursion for x,y explicit *nth* order ODEs

Given $y^{(n)} = f(x, y)$, the forward finite difference recursion for y_k is given by:

$$y_k = h^n f(x_{k-n}, y_{k-n}) + \sum_{i=1}^{n} \binom{n}{i} (-1)^{i+1} y_{k-i}$$

$$\text{with } x_k = x_{k-1} + h, 0 < h \ll 1, \text{and } x_0, y_0, y_1, \cdots, y_{n-1} \text{ known}$$

The values $y_0, y_1, \cdots, y_{n-1}$ are determined from the initial conditions $y_0, y_0', \cdots, y_0^{(n-1)}$ through:

$$y_j = \frac{\left(x_j - x_0\right)^n}{n!} f(x_0, y_0) + \sum_{i=0}^{n-1} \frac{\left(x_j - x_0\right)^i}{i!} y_0^{(i)}$$

1.1.1 | Derivation of x,y forward recursion

Starting with an *nth* order ODE of the form $y^{(n)} = f(x, y)$, the following operators are defined.

Define the shift operator E such that $E^j y_i = y_{i+j}$, and the forward difference operator $\Delta = E - 1$.

Let $y^{(n)}(x_i) = y_i^{(n)}$, and $x_i = x_{i-1} + h$.

Now, $y^{(n)}(x_{k-n})$ can be expressed in terms of Δ using the definition of the derivative:

$$y^{(n)}(x_{k-n}) = y_{k-n}^{(n)} = \frac{1}{h}\left(y_{k-n+1}^{(n-1)} - y_{k-n}^{(n-1)}\right) \text{ , } 0 < h \ll 1$$

$$y_{k-n}^{(n)} = \frac{1}{h} \Delta y_{k-n}^{(n-1)}$$

$$y_{k-n}^{(n)} = \frac{1}{h^n} \Delta^n y_{k-n}$$

Recall that $\Delta = E - 1$, so $\Delta^n = (E-1)^n$, which can be expanded by the binomial theorem:

$$\Delta^n = (E-1)^n = \sum_{j=0}^{n} \binom{n}{j} E^j (-1)^{n-j}$$

Substituting the expression above for Δ^n, and recalling that $y_{k-n}^{(n)} = f(x_{k-n}, y_{k-n})$:

$$f(x_{k-n}, y_{k-n}) = \frac{1}{h^n} \sum_{j=0}^{n} \binom{n}{j} (-1)^{n-j} E^j y_{k-n}$$

$$h^n f(x_{k-n}, y_{k-n}) = \sum_{j=0}^{n} \binom{n}{j} (-1)^{n-j} y_{k-n+j}$$

$$h^n f(x_{k-n}, y_{k-n}) = y_k + \sum_{j=0}^{n-1} \binom{n}{j} (-1)^{n-j} y_{k-n+j}$$

Reindexing with $i = n - j$ and rearranging for y_k:

$$y_k = h^n f(x_{k-n}, y_{k-n}) + \sum_{i=1}^{n} \binom{n}{i} (-1)^{i+1} y_{k-i} \quad \blacksquare$$

1.1.2 | Derivation of initial conditions

As y_k depends on $y_{k-1}, y_{k-2}, \cdots, y_{k-n}$, the first n terms must be known beforehand. This is analogous to an ODE of order n requiring initial conditions for the first n derivatives. The objective is to relate the first n terms of the sequence y_k to the initial conditions of the system, $y_0^{(i)}$.

From Taylor series:

$$y_j = \sum_{i=0}^{\infty} \frac{y_0^{(i)}}{i!} \left(x_j - x_0\right)^i$$

Assuming initial conditions $y_0, y_0', \cdots, y_0^{(n-1)}$ are known, and $y_0^{(n)} = f(x_0, y_0)$ from ODE:

$$y_j = \sum_{i=0}^{n} \frac{y_0^{(i)}}{i!} \left(x_j - x_0\right)^i$$

$$y_j = \frac{y_0^{(n)}}{n!} \left(x_j - x_0\right)^n + \sum_{i=0}^{n-1} \frac{y_0^{(i)}}{i!} \left(x_j - x_0\right)^i$$

$$y_j = \frac{\left(x_j - x_0\right)^n}{n!} f(x_0, y_0) + \sum_{i=0}^{n-1} \frac{\left(x_j - x_0\right)^i}{i!} y_0^{(i)} \quad \blacksquare$$

1.1.3 | First order ODE recursion

With $n = 1$, and initial conditions $\{x_0, y_0\}$, the forward recursion simplifies to Euler's method:

$$y_k = hf(x_{k-1}, y_{k-1}) + y_{k-1}$$
$$x_k = x_{k-1} + h$$

1.1.4 | Second order ODE recursion

With $n = 2$, and initial conditions $\{x_0, y_0, y_0'\}$, the forward recursion becomes:

$$y_k = h^2 f(x_{k-2}, y_{k-2}) + 2y_{k-1} - y_{k-2}$$
$$x_k = x_{k-1} + h$$

$$\text{with } y_1 = y_0 + hy_0' + \frac{h^2}{2} f(x_0, y_0)$$

1.1.5 | Third order ODE recursion

With $n = 3$, and initial conditions $\{x_0, y_0, y_0', y_0''\}$:

$$y_k = h^3 f(x_{k-3}, y_{k-3}) + 3y_{k-1} - 3y_{k-2} + y_{k-3}$$
$$x_k = x_{k-1} + h$$

$$\text{with } y_1 = y_0 + hy_0' + \frac{h^2}{2} y_0'' + \frac{h^3}{6} f(x_0, y_0)$$

$$\text{and } y_2 = y_0 + 2hy_0' + 2h^2 y_0'' + \frac{4h^3}{3} f(x_0, y_0)$$

1.2 | Forward finite difference recursion for general explicit *nth* order ODEs

Given $y^{(n)} = f\big(x, y, y', \cdots, y^{(n-1)}\big)$, the forward finite difference recursion for y_k is now given by:

$$y_k = h^n f\bigg(x_{k-n}, y_{k-n}, \frac{1}{h}\Delta y_{k-n}, \cdots, \frac{1}{h^{n-1}}\Delta^{n-1} y_{k-n}\bigg) + \sum_{i=1}^{n}\binom{n}{i}(-1)^{i+1} y_{k-i}$$

$$\text{with } \Delta^m y_{k-n} = \sum_{i=0}^{m}\binom{m}{i}(-1)^{m-i} y_{k-n+i} \text{ and } x_k = x_{k-1} + h, 0 < h \ll 1$$

The values $y_0, y_1, \cdots, y_{n-1}$ are again determined from the initial conditions $y_0, y_0', \cdots, y_0^{(n-1)}$ through:

$$y_j = \frac{\big(x_j - x_0\big)^n}{n!} f\Big(x_0, y_0, y_0', \cdots, y_0^{(n-1)}\Big) + \sum_{i=0}^{n-1}\frac{\big(x_j - x_0\big)^i}{i!} y_0^{(i)}$$

1.2.1 | Derivation of general forward recursion

Now starting with the general *nth* order ODE of the form $y^{(n)} = f\big(x, y, y', \cdots, y^{(n-1)}\big)$, similarly to the x,y recursion, the following operators are defined.

Define the shift operator E such that $E^j y_i = y_{i+j}$, and the forward difference operator $\Delta = E - 1$.

Let $y^{(n)}(x_i) = y_i^{(n)}$, and $x_i = x_{i-1} + h$. From **1.1.1**:

$$y_{k-n}^{(n)} = \frac{1}{h^n}\Delta^n y_{k-n} \ , \ 0 < h \ll 1$$

Expanding $\Delta^n = (E-1)^n$, and recalling that $y_{k-n}^{(n)} = f\Big(x_{k-n}, y_{k-n}, y_{k-n}', \cdots, y_{k-n}^{(n-1)}\Big)$:

$$f\Big(x_{k-n}, y_{k-n}, y_{k-n}', \cdots, y_{k-n}^{(n-1)}\Big) = \frac{1}{h^n}\sum_{j=0}^{n}\binom{n}{j}(-1)^{n-j} E^j y_{k-n}$$

$$h^n f\Big(x_{k-n}, y_{k-n}, y_{k-n}', \cdots, y_{k-n}^{(n-1)}\Big) = \sum_{j=0}^{n}\binom{n}{j}(-1)^{n-j} y_{k-n+j}$$

$$h^n f\Big(x_{k-n}, y_{k-n}, y_{k-n}', \cdots, y_{k-n}^{(n-1)}\Big) = y_k + \sum_{j=0}^{n-1}\binom{n}{j}(-1)^{n-j} y_{k-n+j}$$

Reindexing with $i = n - j$ and rearranging for y_k:

$$y_k = h^n f\Big(x_{k-n}, y_{k-n}, y_{k-n}', \cdots, y_{k-n}^{(n-1)}\Big) + \sum_{i=1}^{n}\binom{n}{i}(-1)^{i+1} y_{k-i}$$

Then, using $y_{k-n}^{(m)} = \frac{1}{h^m}\Delta^m y_{k-n}$ to eliminate derivatives from f:

$$y_k = h^n f\bigg(x_{k-n}, y_{k-n}, \frac{1}{h}\Delta y_{k-n}, \cdots, \frac{1}{h^{n-1}}\Delta^{n-1} y_{k-n}\bigg) + \sum_{i=1}^{n}\binom{n}{i}(-1)^{i+1} y_{k-i} \quad \blacksquare$$

$$\text{with } \Delta^m y_{k-n} = \sum_{i=0}^{m}\binom{m}{i}(-1)^{m-i} y_{k-n+i}$$

1.2.2 | Second order general ODE recursion

For $n = 2$:

$$y_k = h^2 f\bigg(x_{k-2}, y_{k-2}, \frac{1}{h}(y_{k-1} - y_{k-2})\bigg) + 2y_{k-1} - y_{k-2}$$

2 | Real World Differential Equations

This chapter will derive and study some real world differential equations, with the goal of approximating solutions using the numerical methods outlined in **1**.

2.1 | The *n* body problem

Consider n objects of mass m_i in free space, with only gravity acting between them. The coupled system of second order differential equations that describe each object's motion are:

$$\tilde{x}_i''(t) = \sum_{j \neq i} G m_j \cdot \frac{\tilde{r}_{i,j}}{\left|\tilde{r}_{i,j}\right|^3} \text{ , for } i, j = 1, 2, \cdots, n$$
$$\text{with } \tilde{r}_{i,j} = \tilde{x}_j(t) - \tilde{x}_i(t)$$

2.1.1 | Derivation of the *n* body problem coupled system

To derive the coupled system of differential equations for the n body problem, the following two laws are required:

$$\sum F = ma \text{ , and } F_g = G\frac{m_1 m_2}{r^2}$$

For object i, the following notations are defined:

$m_i = \text{mass of } i$

$\tilde{x}_i(t) = \text{position vector of } i \text{ at time } t$

From Newton's second law of motion:

$$\sum \tilde{F}_i = m_i \tilde{x}_i''(t)$$

From Newton's law of universal gravitation, the magnitude of the force acting on object a due to object b is:

$$\left|\tilde{F}_g^{a,b}\right| = G\frac{m_a m_b}{\left|\tilde{r}_{a,b}\right|^2} \text{ , where } \tilde{r}_{a,b} = \tilde{x}_b(t) - \tilde{x}_a(t).$$

This force always acts towards object b, so:

$$\tilde{F}_g^{a,b} = \alpha(\tilde{x}_b(t) - \tilde{x}_a(t)) = \alpha \cdot \tilde{r}_{a,b}$$

The magnitude of this vector must be equivalent to the expression above, allowing α to be derived:

$$\left|\tilde{F}_g^{a,b}\right| = \alpha\left|\tilde{r}_{a,b}\right|$$
$$G\frac{m_a m_b}{\left|\tilde{r}_{a,b}\right|^2} = \alpha\left|\tilde{r}_{a,b}\right|$$
$$\Rightarrow \alpha = G\frac{m_a m_b}{\left|\tilde{r}_{a,b}\right|^3}$$

Therefore, we have that the gravitational force acting on a due to b is as follows:

$$\tilde{F}_g^{a,b} = G\frac{m_a m_b}{\left|\tilde{r}_{a,b}\right|^3} \cdot \tilde{r}_{a,b}$$

As the objects are in empty space, the only forces acting on them are the gravitational forces from other objects. So, combining Newton's second law and the law of universal gravitation:

$$\sum \tilde{F}_i = \sum_{j \neq i} \tilde{F}_g^{i,j}$$

$$m_i \tilde{x}_i''(t) = \sum_{j \neq i} G \frac{m_i m_j}{\left|\tilde{r}_{i,j}\right|^3} \cdot \tilde{r}_{i,j}$$

$$\tilde{x}_i''(t) = \sum_{j \neq i} G m_j \cdot \frac{\tilde{r}_{i,j}}{\left|\tilde{r}_{i,j}\right|^3} \quad \blacksquare$$

2.1.2 | The discrete three body problem

For $n = 3$, the coupled system becomes:

$$\tilde{x}_1''(t) = G m_2 \cdot \frac{\tilde{r}_{1,2}}{\left|\tilde{r}_{1,2}\right|^3} + G m_3 \cdot \frac{\tilde{r}_{1,3}}{\left|\tilde{r}_{1,3}\right|^3}$$

$$\tilde{x}_2''(t) = G m_1 \cdot \frac{\tilde{r}_{2,1}}{\left|\tilde{r}_{2,1}\right|^3} + G m_3 \cdot \frac{\tilde{r}_{2,3}}{\left|\tilde{r}_{2,3}\right|^3}$$

$$\tilde{x}_3''(t) = G m_1 \cdot \frac{\tilde{r}_{3,1}}{\left|\tilde{r}_{3,1}\right|^3} + G m_2 \cdot \frac{\tilde{r}_{3,2}}{\left|\tilde{r}_{3,2}\right|^3}$$

$$\text{with } \tilde{r}_{i,j} = \tilde{x}_j(t) - \tilde{x}_i(t)$$

The goal is to approximate the solutions to this coupled system. Similar to the forward finite difference derived in **1.1**, $\tilde{x}_i''(t)$ can be expressed as a discrete recursion in k.

Let $t_k = t_{k-1} + h, 0 < h \ll 1$. The second difference for $\tilde{x}_i''(t)$ is:

$$\tilde{x}_i''(t) = \frac{\tilde{x}_i(t+h) - 2\tilde{x}_i(t) + \tilde{x}_i(t-h)}{h^2}$$

To centre the second difference, let $t = t_{k-1}$:

$$\tilde{x}_i''(t_{k-1}) = \frac{\tilde{x}_i(t_k) - 2\tilde{x}_i(t_{k-1}) + \tilde{x}_i(t_{k-2})}{h^2}$$

$$\Rightarrow \tilde{x}_i(t_k) = h^2 \tilde{x}_i''(t_{k-1}) + 2\tilde{x}_i(t_{k-1}) - \tilde{x}_i(t_{k-2})$$

Substituting in the ODE expression for $\tilde{x}_i''(t_{k-1})$:

$$\tilde{x}_i(t_k) = h^2 \sum_{j \neq i} G m_j \cdot \frac{\tilde{r}_{i,j}(t_{k-1})}{\left|\tilde{r}_{i,j}(t_{k-1})\right|^3} + 2\tilde{x}_i(t_{k-1}) - \tilde{x}_i(t_{k-2})$$

$$\text{with } \tilde{r}_{i,j}(t) = \tilde{x}_j(t) - \tilde{x}_i(t)$$

Therefore, the system of recursions for the 3 objects is:

$$\tilde{x}_1(t_k) = h^2\left(Gm_2 \cdot \frac{\tilde{r}_{1,2}(t_{k-1})}{\left|\tilde{r}_{1,2}(t_{k-1})\right|^3} + Gm_3 \cdot \frac{\tilde{r}_{1,3}(t_{k-1})}{\left|\tilde{r}_{1,3}(t_{k-1})\right|^3}\right) + 2\tilde{x}_1(t_{k-1}) - \tilde{x}_1(t_{k-2})$$

$$\tilde{x}_2(t_k) = h^2\left(Gm_1 \cdot \frac{\tilde{r}_{2,1}(t_{k-1})}{\left|\tilde{r}_{2,1}(t_{k-1})\right|^3} + Gm_3 \cdot \frac{\tilde{r}_{2,3}(t_{k-1})}{\left|\tilde{r}_{2,3}(t_{k-1})\right|^3}\right) + 2\tilde{x}_2(t_{k-1}) - \tilde{x}_2(t_{k-2})$$

$$\tilde{x}_3(t_k) = h^2\left(Gm_1 \cdot \frac{\tilde{r}_{3,1}(t_{k-1})}{\left|\tilde{r}_{3,1}(t_{k-1})\right|^3} + Gm_2 \cdot \frac{\tilde{r}_{3,2}(t_{k-1})}{\left|\tilde{r}_{3,2}(t_{k-1})\right|^3}\right) + 2\tilde{x}_3(t_{k-1}) - \tilde{x}_3(t_{k-2})$$

$$\text{with } \tilde{r}_{i,j}(t) = \tilde{x}_j(t) - \tilde{x}_i(t) \text{ and } t_k = t_{k-1} + h, 0 < h \ll 1$$

To start the recursion at t_2, both $\tilde{x}_i(t_0)$ and $\tilde{x}_i(t_1)$ are required to be known. $\tilde{x}_i(t_1)$ can be expressed in terms of the initial velocity, $\tilde{x}_i'(t_0)$ through Taylor series:

$$\tilde{x}_i(t_1) = \tilde{x}_i(t_0) + h\tilde{x}_i'(t_0) + \frac{h^2}{2}\tilde{x}_i''(t_0)$$

2.1.3 | Simulating the three body problem

With the discrete recursions derived above, the coupled system can be simulated in Python. After defining the initial conditions and parameters of the system, the core recursion loop is:

```
    x1n = h ** 2 * G * (m2 * (x2[1] - x1[1]) / np.linalg.norm(x2[1] - x1[1]) ** 3 +
m3 * (x3[1] - x1[1]) / np.linalg.norm(x3[1] - x1[1]) ** 3) + 2 * x1[1] - x1[0]

    x2n = h ** 2 * G * (m1 * (x1[1] - x2[1]) / np.linalg.norm(x1[1] - x2[1]) ** 3 +
m3 * (x3[1] - x2[1]) / np.linalg.norm(x3[1] - x2[1]) ** 3) + 2 * x2[1] - x2[0]

    x3n = h ** 2 * G * (m1 * (x1[1] - x3[1]) / np.linalg.norm(x1[1] - x3[1]) ** 3 +
m2 * (x2[1] - x3[1]) / np.linalg.norm(x2[1] - x3[1]) ** 3) + 2 * x3[1] - x3[0]
```

These compute the next position vector of each object, with `np.linalg.norm()` calculating the magnitude of the argument vector. Due to the recursion requiring the two previous position vectors, the new positions are stored as follows:

```
x1[0] = x1[1]
x1[1] = x1n

x2[0] = x2[1]
x2[1] = x2n

x3[0] = x3[1]
x3[1] = x3n
```

That is, the stored vectors update as:

$$[\tilde{x}_i(t_{k-2}), \tilde{x}_i(t_{k-1})] \to [\tilde{x}_i(t_{k-1}), \tilde{x}_i(t_k)]$$

Lastly, the new position vectors are plotted using `pyplot`, simulating the three body problem.

2.2 | Pendulum dynamics

Consider a point mass m attached to a massless, rigid rod of length l, swinging from a pivot.

Gravity and air resistance act on the mass.

If $\theta(t)$ is the angle that the rod makes with the vertical at time t, then the dynamics of $\theta(t)$ are:

$$\theta''(t) = -\frac{g}{l}\sin\theta(t) - \frac{\mu}{ml^2}\theta'(t)$$

2.2.1 | The discrete damped pendulum

To numerically approximate the damped pendulum, the ODE is again turned into a recursion, as seen in **1.2** and **2.1.2**.

Let $t_k = t_{k-1} + h, 0 < h \ll 1$. The forward finite difference for $\theta'(t)$ and $\theta''(t)$ is:

$$\theta'(t) = \frac{\theta(t+h) - \theta(t-h)}{2h}$$
$$\theta''(t) = \frac{\theta(t+h) - 2\theta(t) + \theta(t-h)}{h^2}$$

Let $t = t_{k-1}$ and $\theta(t_n) = \theta_n$:

$$\theta'_{k-1} = \frac{\theta_k - \theta_{k-2}}{2h}$$
$$\theta''_{k-1} = \frac{\theta_k - 2\theta_{k-1} + \theta_{k-2}}{h^2}$$

Substituting into dynamics equation:

$$\frac{\theta_k - 2\theta_{k-1} + \theta_{k-2}}{h^2} = -\frac{g}{l}\sin\theta_{k-1} - \frac{\mu}{ml^2}\cdot\frac{\theta_k - \theta_{k-2}}{2h}$$

Solving for θ_k and simplifying:

$$\theta_k = \frac{2ml(2l\cdot\theta_{k-1} - h^2 g\cdot\sin\theta_{k-1}) + (h\mu - 2ml^2)\cdot\theta_{k-2}}{h\mu + 2ml^2}$$
$$\text{with } t_k = t_{k-1} + h, 0 < h \ll 1$$

Likewise from **2.1.2**, to begin the recursion at t_2, both θ_0 and θ_1 are required to be known. Assuming that the initial velocity $\theta'_0 = 0$, and using Taylor expansion:

$$\theta_1 = \theta_0 + h\theta'_0 + \frac{h^2}{2}\theta''_0$$
$$\theta_1 = \theta_0 - \frac{h^2 g}{2l}\sin\theta_0$$